COM6012 Scalable Machine Learning Assignment

Student registration number: 250117677

1 Log Mining and Analysis

From the NASA access log data for July 1995, the following hosts are filtered: UK academic institutions, UK companies, and UK government departments. For each, the total number of requests and the number of unique hostnames is determined, as shown in Table 1.

Table 1: Total requests and unique hostnames by UK sector.

Sector	Total Requests	Unique Hosts
Academic	25009	1022
Company	22225	1116
Government	195	15

Each institution domain is extracted by first splitting each hostname using the full stop delimiter. For example, the Sheffield hostname, `www.shef.ac.uk`, is split into `["www", "shef", "ac", "uk"]`. The final three labels are concatenated together, resulting in `shef.ac.uk`, which represents the institution domain. Different hosts from the same institution (`cs.shef.ac.uk`, `www.shef.ac.uk`, et cetera) are grouped together, allowing requests to be aggregated at the institution level. The implementation of this method in Spark is demonstrated in Listing 1.

Listing 1: Institution domain extraction and aggregation applied to UK academic hosts. The `extract_domain` function can be applied to other hostname suffixes.

```
def extract_domain(data, suffix: str, alias: str):

    filtered_hosts = data.filter(col("host").endswith(suffix))

    split_host = split(col("host"), "\\.")

    institution_domains = filtered_hosts.select(
        col("host"),
        col("timestamp"),
        concat_ws(".",
                element_at(split_host, -3),
                element_at(split_host, -2),
                element_at(split_host, -1)
        ).alias(alias)
    )

    return institution_domains

academic_domains = extract_domain(logs, ".ac.uk", "institution")

academic_counts = academic_domains \
    .groupBy("institution") \
    .count() \
    .withColumnRenamed("count", "total_requests")
```

The total number of requests from the University of Sheffield domain is 623. There are seven

institution domains with more requests than Sheffield, whose total requests are shown in Table 2.

Table 2: UK academic institution domains with more requests than the University of Sheffield domain.

Institution Domain	Total Requests
`hensa.ac.uk`	4257
`rl.ac.uk`	1158
`ucl.ac.uk`	1036
`man.ac.uk`	921
`ic.ac.uk`	851
`soton.ac.uk`	808
`bham.ac.uk`	629

The nine most active UK company (`.co.uk`) hosts and their respective total requests are displayed in Figure 1. A bar chart is chosen to compare request counts across discrete company categories. It clearly shows the top nine companies alongside all remaining companies combined, using contrasting colours to highlight the comparison and improve readability.

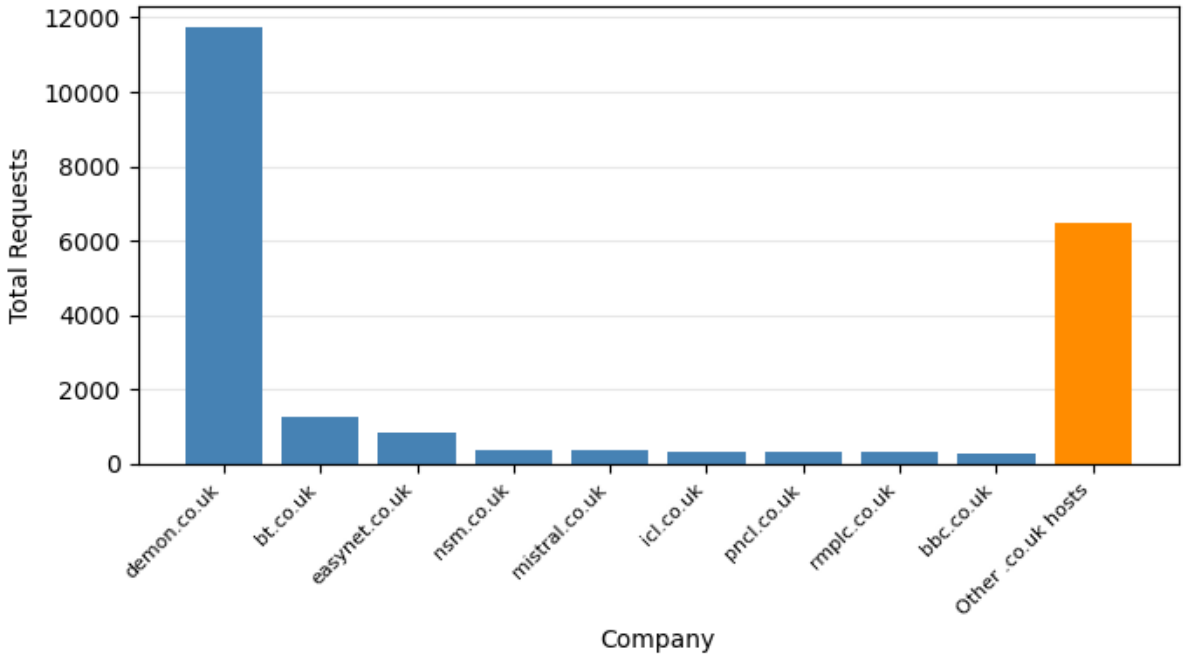
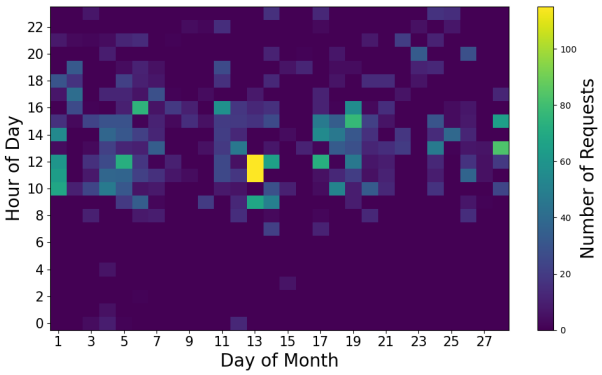


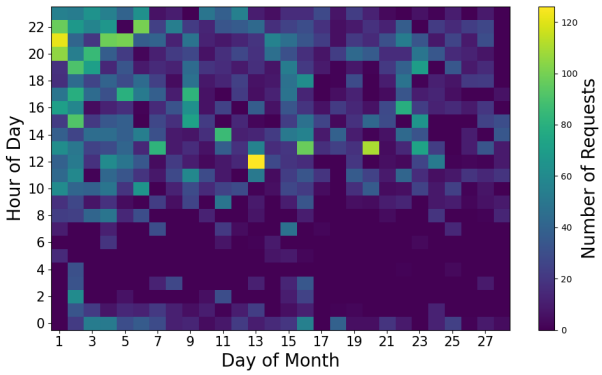
Figure 1: Total requests from the nine most active UK company domains, compared with the combined total for all remaining UK companies.

The most active academic institution is `hensa.ac.uk` with 4 257 requests, while the most active company domain is `demon.co.uk` with 11 719 requests. Figure 2 shows the hourly request patterns for these domains and all `.gov.uk` hosts combined. The heatmaps show that access activity is typically concentrated during daytime and evening hours. The `demon.co.uk` heatmap displays the densest activity overall, whereas the combined `.gov.uk` requests are much more infrequent.

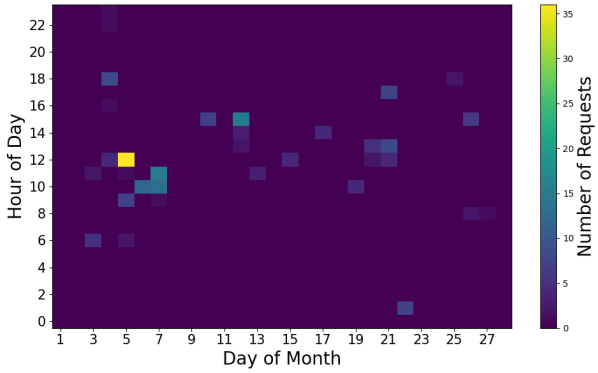
Observation 1: The UK academic and company domains are much more active than the UK government sector, as shown in Table 1. This may be because NASA resources are more



(a) `hensa.ac.uk` access pattern.



(b) `demon.co.uk` access pattern.



(c) `.gov.uk` sector access pattern.

Figure 2: Hourly access patterns across days 1–28 for the most active UK academic institution domain (a), company domain (b), and all UK government domains combined (c). Colour intensity represents the number of requests.

relevant to research, education, and commercial users than to UK government departments. This is useful to NASA as it highlights the main groups accessing its online resources, allowing things like server capacity to be focused on the domains generating the most demand.

Observation 2: The company requests are highly concentrated in one domain (`demon.co.uk`), which has significantly more requests than any other individual company domain shown in Figure 1. Between 1992–1997, Demon Internet was a British internet service provider with the domain name `demon.co.uk`. Therefore, requests from many different users of their service may appear under the same company domain. This is useful to NASA as it shows that large numbers of requests from a single domain might reflect an access provider, rather than the behaviour of one individual user group.

2 Scalable Logistic Regression and Generalised Linear Models

The UK traffic vehicle-count dataset is preprocessed by extracting month and weekday features, converting travel directions to uppercase, removing missing target values, one-hot encoding categorical variables, and standardising numerical features. The processed dataset is then split by year into training, validation and test sets, with sample sizes and corresponding date ranges shown in Table 3.

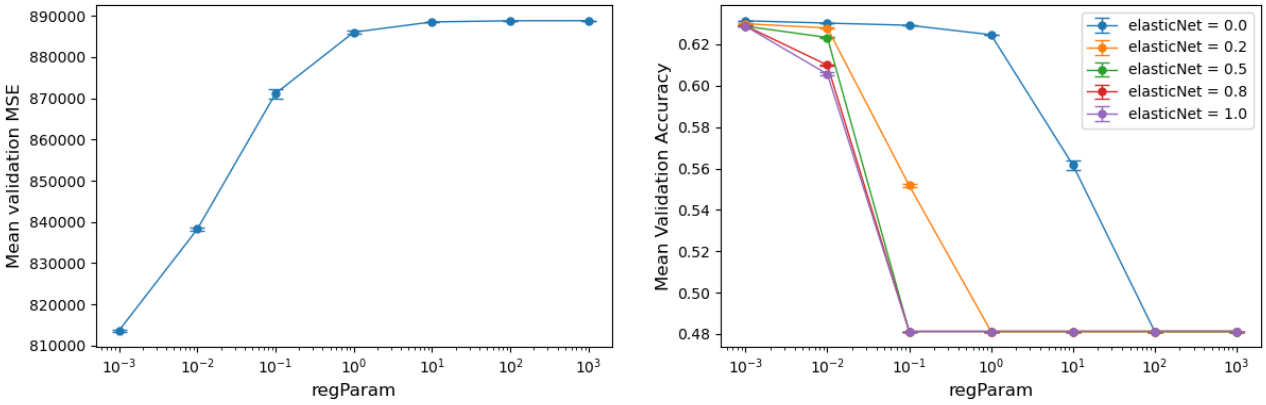
A Poisson regression model uses the training and validation sets to tune the `regParam` hyper-

Table 3: Sample sizes of training, validation and test sets

Set	Years	Sample Size
Training	2000–2021	4655073
Validation	2022–2023	306804
Test	2024	151848

parameter over the values [0.001, 0.01, 0.1, 1, 10, 100, 1000], each with five random subsets that sample 80% of the training set. The random seeds used are 17677, 17678, 17679, 17680, and 17681 (using student registration number 250117677).

The resulting mean validation MSE of each `regParam` value is shown in Figure 3a. The optimal value is `regParam` = 0.001, which has the lowest mean validation MSE (813 706.18 ± 237.58). Increasing `regParam` from 0.001 to 0.01 and 0.1 causes a sharp increase in MSE, which plateaus for larger values. This suggests that stronger regularisation causes the Poisson regression model to underfit.



(a) Mean validation MSE for Poisson regression across values of `regParam`.

(b) Mean validation accuracy for logistic regression across values of `regParam` and `elasticNetParam`.

Figure 3: Validation curves for the regression models. Error bars show ± one standard deviation across five random subsets, each sampling 80% of the training set.

Using the optimal value of `regParam` = 0.001, the model is retrained on the combined training and validation sets. The final model is tested on the test set, achieving MSE = 787 575.11.

The learned coefficient vector is reported in Appendix A.

The median value of `all_motor_vehicles` in the training set is 235.0. This value is used as a threshold between low and high traffic, allowing a binary target column, `traffic`, to be created. A logistic regression model is then trained on the processed training dataset. The same `regParam` values as in Figure 3a are tested, as well as the `elasticNetParam` values [0.0, 0.2, 0.5, 0.8, 1.0]. Five random training subsets are created to record the mean validation accuracy for each combination of `regParam` and `elasticNetParam`, which is shown in Figure 3b.

Discussion of figure here.

Using the optimal `regParam` and `elasticNetParam`, the logistic regression model is retrained on the combined training and validation sets. It is then evaluated on the test set, achieving an accuracy of 0.6404. The final learned model coefficients are reported in Appendix A.

One observation that you find interesting.

3 Scalable Decision Trees, Random Forests, and Gradient Boosting

Stratified sampling is used to create a 2% subset of the HIGGS dataset. To optimise the Random Forest and Gradient-Boosted Tree (GBT) models, 3-fold cross-validation is performed on this subset. The chosen hyperparameters, their tested values, and the resulting optimal values are shown in Table 4. The subset is then split into training and test sets which are used

Table 4: Parameter grids and optimal values selected by 3-fold cross-validation of the Random Forest and Gradient-Boosted Tree models on the sampled HIGGS dataset.

Model	Hyperparameter	Tested Values	Optimal Value
Random Forest	`maxBins`	`[16, 32, 64]`	64
	`maxDepth`	`[3, 5, 7]`	7
	`numTrees`	`[10, 20, 40]`	20
Gradient-Boosted Tree	`maxBins`	`[16, 32, 64]`	32
	`maxDepth`	`[3, 5, 7]`	7
	`maxIter`	`[10, 20, 40]`	40

to evaluate the performance of both models, which is reported in Table 5.

Table 5: Training and test accuracy of the tuned models on the sampled HIGGS dataset.

Model	Training Accuracy	Test Accuracy
Random Forest	0.701	0.692
Gradient-Boosted Tree	0.747	0.720

The final performance of both models is evaluated using the full HIGGS dataset, which is split into training and test sets of 7 698 149 and 3 301 851 samples, respectively. Each model is trained using the optimal hyperparameters found in Table 4 and evaluated on the test set. Table 6 shows the resulting performance of each model which, in both cases, is very similar on both training and test sets. Compare performance on both training and test sets between the

Table 6: Training and test accuracy of the tuned models on the full HIGGS dataset.

Model	Training Accuracy	Test Accuracy
Random Forest	0.692	0.691
Gradient-Boosted Tree	0.726	0.726

two models.

4 Movie Recommendations and Cluster Analysis

Before model training, the movie ratings are sorted chronologically using the timestamp column, then split into training and test sets. Three training size splits are considered: 40%, 60%, and

80%, each using earlier times for training and later times for testing. For each split, two ALS settings are studied. The first uses the default PySpark ALS configuration from Lab 6, with `userCol` = `"userId"`, `itemCol` = `"movieId"`, `ratingCol` = `"rating"`, `seed` = 250117677, and `coldStartStrategy` = `"drop"`. The second setting uses `rank` = 25, allowing the model to represent more latent user-movie preference patterns, together with `regParam` = 0.15, which helps to reduce overfitting as model complexity increases. Finally, `maxIter` = 10 is used to give the ALS optimisation more opportunity to converge.

The performance of the two ALS settings on each training split size is shown in Table 7. The results show that the second setting actually performs worse than the first. (CHANGE "ACTUALLY")

Table 7: Recommendation performance for the two ALS settings across chronological train-test splits.

	ALS Setting 1			ALS Setting 2		
Training Split	RMSE	MSE	MAE	RMSE	MSE	MAE
40%	0.803	0.645	0.616	0.822	0.675	0.635
60%	0.777	0.603	0.591	0.792	0.628	0.605
80%	0.797	0.635	0.604	0.810	0.656	0.617

The user factor vectors learned from the second ALS setting are clustered using PySpark k-means with $k = 25$. The five largest clusters for each training split and their number of users are reported in Table 8. The cluster sizes increase with training split size, which is consistent with larger training sets containing more users with learned ALS factor vectors. However, within each split the five largest clusters are relatively similar in size.

Table 8: Top five user-cluster sizes for each chronological training split using ALS Setting 2 user factors and k-means with $k = 25$.

	Cluster Size				
Training Split	Rank 1	Rank 2	Rank 3	Rank 4	Rank 5
40%	4543	4373	4353	4291	4052
60%	5975	5968	5941	5881	5565
80%	8455	7990	7866	7002	6699

Considering all users in the largest cluster of each training split, the top movies are defined as those with an average rating ≥ 4. The genres associated with these movies are counted, allowing multiple genre contributions per movie. Table 9 shows the ten most popular genres for each training split. Across all splits, drama and comedy are the two most common genres among highly rated movies in the largest cluster. The remaining genres are also consistent, with genres like romance, thriller, and crime appearing across all splits.

Observation 1: The second ALS setting demonstrates worse performance for all three training splits, despite being designed to improve upon the first ALS setting. This could be due to the increased `rank` parameter making the model more complex, with insufficient values of `regParam` and `maxIter` to improve generalisation on future ratings. For a movie website such as Netflix, this shows that more complex recommendation models are not necessarily better, and that model changes should be carefully evaluated before deployment.

Table 9: Top ten most popular genres among highly rated movies in the largest user cluster for each chronological training split.

	40% Split		60% Split		80% Split	
Rank	Genre	Count	Genre	Count	Genre	Count
1	Drama	449	Drama	167	Drama	591
2	Comedy	211	Comedy	62	Comedy	293
3	Romance	125	Romance	34	Romance	178
4	Thriller	112	Documentary	34	Thriller	138
5	Crime	101	Thriller	31	Action	128
6	Action	87	Crime	30	Crime	123
7	Adventure	80	War	29	Adventure	110
8	War	75	Adventure	28	Documentary	102
9	Documentary	52	Action	25	War	96
10	Mystery	50	Mystery	19	Mystery	59

Observation 2: Drama and comedy are the two most common genres among highly rated movies in the largest cluster for all three training splits, as shown in Table 9. This could be because they are broad umbrella genres and could be attributed to smaller sub-genres such as dark comedy, romantic comedy, et cetera. It may also indicate the presence of mainstream preferences in the largest user group, rather than highly specialised taste profiles. This could be useful to a movie site such as Netflix in identifying general audience preferences and prioritising recommendations that are broadly appealing. Small clusters could still be used to provide more personalised suggestions.

Appendix

Learned model coefficients.